

12

Verification

What counts as evidence, and how much verification each change needs

Every change you accept gets some amount of verification. Left unplanned, the amount tracks the time and attention available on the day rather than the risk of the change, and the mismatch is worst for the most expensive failures, which are usually the silent ones. This chapter gives you a small system: what counts as evidence, four levels of it, and a rule for choosing the level per change from what the change touches.

The model

A claim is not evidence

When your assistant says “I’ve added policies to all tables” or “it handles errors now,” you’ve received a report from the party that did the work. The report may well be true. It’s still a report, and the move this chapter is built on is converting reports into things you can observe: “policies on all tables” becomes a count you read off a list; “handles errors now” becomes a request you block on purpose and a screen you look at.

Without reading code, three forms of evidence are available to you:

1. **Behaviour you observe** — you run the request, you look at the screen, you read the rows.
2. **Assertions you execute** — checks the assistant writes and you run, which pass or fail in front of you.
3. **Output you inspect** — a list, a count, a diff summary whose shape you can judge.

Everything outside these three is the assistant reporting on itself.

What an assertion is

A **test** is a small program that performs one action against the code and checks one claim about the result: call the function with these inputs and the output must equal this; request user A’s row as user B and the result must be empty. It passes or fails, and it can be run again — after every change, on every deploy — which is what separates it from the first form. Evidence you produce by hand is true at the moment you produce it; an assertion keeps being true, or stops and says so.

The property that matters most is one you can judge without reading the test: could it fail? A real test asserts a specific claim, and a change that broke the claim would make it fail. A test that passes on an empty result, that checks the code ran without checking what it did, or that asserts a value the test itself set up, can’t fail on the claim it’s named after — the proxy green from the entries below, written down as a test.

The going-deeper section has the ask for this, with “what change would make it fail” as the completeness criterion.

Reports from the author

An account of work written by the party that did the work is shaped by the assumptions that produced the work. This is older than any particular tool — it’s why code review is assigned to someone other than the author, why audits are external, and why your own last-month decisions look sounder in memory than in the file. An assistant is no exception: the session that wrote the code holds the context that made every choice seem right, and asked “is this okay?” it tends to say yes.

With an assistant, an independent reviewer is cheap. A fresh session with the code pasted in cold, a framing that asks for problems rather than approval, or a different assistant each surface more findings than asking the author. How much more varies by tool and version, so measure it on your own setup rather than assuming — the going-deeper section has the experiment, and re-running it when your tool changes is what keeps this chapter calibrated as tools improve.

The four levels

LEVEL	WHAT IT MEANS	COST	LASTS
1	Evidence you produce yourself — run a request, read the rows, look at the screen	Minutes	That moment
2	Assertions the assistant writes and you execute, which pass or fail	Minutes, once written	Every run after
3	The assistant’s report on its own work, plus a spot check	Seconds	No
4	Nothing	None	—

The levels are ordered by how much of the evidence you produced yourself, which is also their cost order, and the order says nothing about strength: a level-1 check is the most direct evidence and the least durable, since nothing re-runs it, and a level-2 assertion is what keeps a claim true after the day you checked it. The two are complements — the first tells you a claim holds now, the second tells you when it stops.

Nobody works at level 1 for everything, and nobody should — most changes don’t warrant it, which is why shipping fast is possible at all. The skill is matching the level to the change.

Risk tiers

The level is decided by what the change touches, in advance — not by how the change went, how big it looked, or how the session felt.

TIER	APPLIES TO	LEVEL
A	Authorization, money, deletion, migrations, secrets, anything crossing the trust boundary	1 at the change, then 2 kept — evidence you run, and an assertion that keeps running
B	Business rules, state machines, calculations, anything with a correct answer	2 — assertions you execute
C	Interface states, styling, copy, internal tooling	3 — report plus spot check
D	Prototypes, throwaway scripts, local experiments	4 — nothing

WHICH LEVEL EACH TIER GETS

		level 1 you run it	level 2 assertions	level 3 report, spot check	level 4 nothing
tier A	authorization, money, deletion, migrations, secrets	at the change	kept running		
tier B	rules, state machines, calculations				
tier C	interface, styling, copy, tooling				
tier D	prototypes, throwaway scripts				

tiers B–D relax as your measured reliability improves; tier A doesn't, since its failures are silent

Three refinements carry most of the practical weight:

A change spans tiers at the highest tier it touches. A styling pass that also edits a migration is a tier A change with some styling in it.

Better tools relax tiers B through D, by measurement rather than by feel. As generation gets more reliable, a spot check can replace assertions wherever your own measured failure rate supports it, the same way any process retires an inspection step.

Tier A stays, because its case doesn't rest on the generator's error rate: its failures are silent, so no feedback ever arrives to justify relaxing, and severe, so the check stays cheap relative to the loss. Authorization, money, and migration changes get independent review in all-human teams too, whatever the author's seniority.

Small unreviewed choices compound. Each level-4 acceptance is individually fine; jointly they produce a codebase you no longer know the contents of — at which point you can't ask good questions about it, and the whole method degrades. The answer is scheduled audits for the failures that never produce a symptom on their own: an entry in the calendar, with a stated question to ask, for the things nothing will ever remind you of — schema decisions aging badly, accumulated duplication, cost curves.

Asking for enumerations

The asks throughout this series share one design: they demand a list with a property you can judge without reading its contents. A vague question has no failing answer — “are there any authorization gaps?” returns “this looks good” reliably, because nothing in the question forces a finding to surface. The patterns that work:

1. **List with a completeness criterion** — “every table, all four operations; flag any with fewer than four.” You can count rows and flags without understanding SQL.
2. **State the source** — “for each write, session or request body?” A forced choice between a right answer and a wrong one.
3. **Say none if absent** — the instruction that turns a silent omission into a visible word.
4. **Walk the failure path** — “step by step, what stops this bad request, and does anything actually?” The walk must reach a definite ending.
5. **List what this depends on** — surfaces the neighbour that the change silently leans on.
6. **Plant a control** — before trusting an enumeration, seed one finding you know about and see whether the list surfaces it. A list that misses the planted item wasn't complete, and a list that has never had this done to it has never been shown able to find anything.

And the anti-patterns: yes/no questions, “any problems?”, and anything the assistant can satisfy with a summary. Where you ask matters as much as how — a fresh session with the code pasted in, per the section above.

The asks also assume an assistant that can read your project: a tool that works inside your files rather than a chat window you paste into. From a chat window, every audit starts with pasting the relevant files, and the list it returns covers only what you pasted.

PART

What goes wrong

This chapter’s failures live in the verifying rather than the code, and each entry gives the habit, its tell, and the correction.

1 The vague question

a review request with no failing answer.

INSTEAD one of the enumeration patterns above, with the completeness criterion stated.

TELL *your question could be answered with “looks good” and often is.*

2 Asking the author

the session that wrote the code is the session reviewing it.

INSTEAD fresh session, code pasted cold, “list the problems.”

TELL *“are you sure?” asked downward in the same thread, answered with a defence.*

3 The proxy green

a check that passes while measuring something adjacent to the claim.

INSTEAD name the claim first, then pick the check that would show it false — a check that can’t fail the claim isn’t verifying it.

TELL *the evidence is “it compiles,” “the page loads,” or “the tests pass” when the claim was about behaviour no test covers — or a test that no change to the code could make fail.*

4 Tier A on autopilot

an authorization, money, or migration change verified at level 3 because it was small.

INSTEAD tier by surface, not by size; a one-line policy change is a tier A change of one line.

TELL *“it was a one-liner” as the reason no evidence was produced.*

THE DIRECT TEST

Take the most recent claim your assistant made about its own work. Produce the level-1 evidence for it — run the request, read the rows, look at the screen. If you can’t say what that evidence would consist of, that’s the finding: the claim was operating at level 3 or 4.

PART

Going deeper

These prompts are for your assistant, and each comes with a note on what a good response looks like.

D1 · Tier the history

AUDIT

Look at the last twenty changes in my project’s history. For each: one line on what it touched, its tier — A for authorization/money/deletion/migrations/secrets/trust boundary, B for logic with a correct answer, C for interface and tooling, D for throwaway — and, as far as you can tell, what verification it actually got. Present it as a table and flag every A-row whose verification you can’t point to.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response is a table you can read in one pass, and the flagged rows are your worklist. If tiers come back without reasons, ask for the reason column — “touches a migration” is checkable, a bare “A” isn’t.

D3 · Measure the self-defence

BUILD A TOY

Take a recent substantial change. In the session that produced it, ask: “list the problems with this change.” Then open a fresh session, paste the same change in cold, and ask the identical question. Count the findings in each and compare.

WHAT A GOOD RESPONSE LOOKS LIKE

It takes about twenty minutes and gives you a measurement of what review-by-the-author costs on your own tool at its current version. Re-run it when the tool changes; the measurement is what keeps this chapter’s advice matched to your setup.

D2 · Convert the claims

GROUNDING EXPLANATION

Here are three claims made about recent work on my project: [paste them]. For each, give me a check I can run in under a minute that would show the claim false if it is false. The runnable check itself — what I type or click, and what result means pass. “Re-read the code” doesn’t count.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response gives three checks, each with a stated pass condition and ideally a control — a variant that must fail or must return something, proving the check can detect what it claims to. A check that passes on an empty result proves nothing until you’ve seen it able to fail. Any check you couldn’t hand to someone else to run isn’t finished — re-ask for the exact steps.

D4 · The tier drill

PREDICTION QUIZ

Quiz me on tiering. One at a time, describe a plausible change to my project and ask which tier it is and what verification it should get. Wait for my answer, then correct me with the reason. Include one change that spans tiers, one that looks small but is tier A, and one that sounds scary but is honestly tier C.

WHAT A GOOD RESPONSE LOOKS LIKE

Commit before reading each correction. The spanning case and the small-but-A case are where the misjudgements live.

D5 · Audit my tests

AUDIT

List every test in my project. For each: one line on the claim it checks, and one specific change to the code that would make it fail. Flag every test where you can't name such a change, and every test that passes on an empty result. Then list the tier A and tier B behaviour in my project that no test covers. Full table, no summarizing.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response names a breaking change per test, and the flags are tests that measure something adjacent to their name. The last list is the worklist: any tier A row on it is a claim held at level 1 on the day it was checked and at level 4 every day since.

WHERE THIS CONNECTS

Every *ask* and *check* in this series is this chapter's method applied to one surface — **chapter 6 · Authorization** is the fullest worked example. **Chapter 11 · The loop** is the working cycle this decision sits inside, and its house rules are how a tier decision becomes something your assistant participates in rather than something you remember alone.